

Object Oriented Programming

It is a programming approach that uses **objects and classes** to organize code in a way that makes it easier to develop, maintain, and reuse.

Class & Object

What is a Class?

A **class** is a model or template used to create objects.
It does **not occupy memory until we create an object**.

What is an Object?

An **object** is a Instance of a class or real entity created from a class.
It holds data (variables) and can perform actions (methods).

Real-World Example

- **Class** → **House Blueprint**
- **Object** → **Actual house built using blueprint**
- **Class:** Car
- **Objects:** BMW, Audi, Tesla

Blueprint ≠ Real house

But blueprint helps build many houses → same class can create many objects.

Java Example

```
class Car {  
String model = "Tata Nexon";  
int price = 1000000;
```

```
    void start() {  
        System.out.println("Car Started");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car(); // Object creation
```

```
System.out.println(myCar.model);  
myCar.start();  
}  
}
```



Output

```
Tata Nexon Car Started
```



2

Encapsulation



Easy Explanation

Encapsulation means **protecting data** by keeping it private inside a class and only allowing access through methods.

It provides:

Feature	Meaning
Data Hiding	Prevents direct modification
Security	Only allowed operations can modify values
Control	We can validate input before changing data



Real-World Example

- Bank has your account balance.
- You **cannot directly change money**.
- You must use deposit/withdraw methods.



Java Example

```
class Person {
```

```
    private String name;    // data hidden  
  
    public void setName(String n) {  
        name = n;          // setting value  
    }  
  
    public String getName() {  
        return name;       // getting value  
    }  
}
```

```
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.setName("Aswin"); // set value
        System.out.println(p.getName()); // print value
    }
}
```

Output

Aswin

3 Inheritance

Easy Explanation

Inheritance allows one class (child) to use the properties and methods of another class (parent). We use the `extends` keyword.

Purpose:

- ✓ Code reusability
- ✓ Avoid duplication
- ✓ Better structure

Real-World Example

A child inherits eye color, height from parents.

Java Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal { // Child inherits parent
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

Output

```
Animal is eating Dog is barking
```

Polymorphism

Easy Explanation

Polymorphism means **one thing behaving in many ways**.

Two types:

Type	Meaning	Example
Compile-Time (Overloading)	Same method name, different parameters	print(), print(int), print(double)
Run-Time (Overriding)	Same method name in parent & child but different implementation	sound() in animal, dog, cat

Real-World Example

A smartphone button does different actions:

- single tap → open app
- long press → show options

Same button, different behavior → Polymorphism.

Java Example (Overriding)

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Cat(); // runtime decision  
        a.sound();  
    }  
}
```



Output

```
Cat meows
```



5

Abstraction

Definition

Hiding internal details and showing only necessary information using **abstract classes** or **interfaces**.

Real-World Example

- When driving a car, you use **steering wheel**, not the internal engine mechanism.

Java Example

```
abstract class Car {  
    abstract void start();  
}  
  
class Tesla extends Car {  
    void start() {  
        System.out.println("Tesla starts silently");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c = new Tesla();  
        c.start();  
    }  
}
```

```
}  
}
```

Output

```
Tesla starts silently
```



5

Abstraction



Easy Explanation

Abstraction means **showing only important details** and hiding complex internal working.

It is implemented using:

✓ Abstract Classes

✓ Interfaces

You interact with simple methods while the complex logic is hidden.



Real-World Example

You drive a car using steering, brakes, accelerator but you don't see engine, piston movement.



Java Example

```
abstract class Vehicle {  
    abstract void start(); // no implementation  
}  
  
class Bike extends Vehicle {  
    void start() {  
        System.out.println("Bike starts with kick or button");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Vehicle v = new Bike();  
        v.start();  
    }  
}
```



Output

```
Bike starts with kick or button
```

Final Quick Summary

Concept	Meaning	Real Example
Class	Blueprint	Mobile design
Object	Instance of class	Real mobile
Encapsulation	Protect and control data	ATM Pin
Inheritance	Reuse parent features	Child inherits traits
Polymorphism	One action, many behaviors	Same button, different actions
Abstraction	Hide complexity	Driving car without seeing engine